E-Voting Pilot System: Full Stack Master Technical & Viva Guide

Architecture: Spring Boot 3.4.2 (Java 21 LTS) | Angular 22 Standalone | MySQL 8.x | Spring Cloud Gateway | Eureka Discovery
Prepared for: Academic Project Evaluation, Senior Viva Defense, and Full Stack Technical Interviews

---

1. Spring Boot & Microservices Engineering Topics

Spring Boot Topic	Implementation in E-Voting System	Architectural Rationale
Spring Cloud Netflix Eureka (Port 8761)	Implemented in eureka-server with @EnableEurekaServer. Microservices annotate @EnableDiscoveryClient to register dynamic IP and port bindings.	Eliminates hardcoded hostnames and ports. Enables dynamic location transparency and client-side load balancing.
Spring Cloud API Gateway (Port 8080)	Implemented in api-gateway using reactive non-blocking Spring WebFlux. Configured route predicates (/api/votes/** -> lb://VOTING-SERVICE), global CORS, and JwtAuthenticationFilter.	Serves as single entry point for all client traffic. Centralizes security filtering, rate limiting, and request routing.
Spring Security 6 & JJWT (Port 8086)	Implemented in auth-service using SecurityFilterChain, stateless sessions (SessionCreationPolicy.STATELESS), BCrypt password hashing (strength 12), and JJWT 0.12.6.	Ensures zero backend session stickiness. Cryptographic claims in JWT guarantee tamper-proof user authentication and roles.
Spring Data JPA & Hibernate 6	Implemented across all business microservices using @Entity, @Table, @Id, @Column, @CreationTimestamp, and Spring Data JpaRepository.	Abstracts SQL boilerplate into high-level domain entities with type-safe derived and JPQL queries.
OpenFeign Declarative REST Client	Implemented in voting-service via VoterClient interface annotating @FeignClient(name = "voter-service").	Replaces manual RestTemplate wiring with declarative compile-time interfaces that dynamically resolve target endpoints via Eureka.
Resilience4j Circuit Breakers	Configured in voting-service application.yml with sliding window size of 10, failure rate threshold of 50%, and 10-second open-state	Prevents cascading failures. If voter-service experiences latency, the circuit breaker opens to prevent

	duration.	thread pool exhaustion in voting-service.
Transaction Management (@Transactional)	Applied on VotingService.castVote() and AuditServiceImpl.recordAuditLog().	Enforces ACID atomicity across the balloting transaction. If hash generation or receipt persistence fails, the entire transaction rolls back cleanly.
Centralized Exception Handling	Implemented using @RestControllerAdvice and @ExceptionHandler across microservices to generate uniform JSON error responses.	Masks internal database stack traces from leaking to clients while returning standardized HTTP status codes (400, 401, 404, 409).
Jakarta Bean Validation	Applied on request DTOs using @NotBlank, @Size, @NotNull, and @Valid in controller endpoints.	Validates incoming payload integrity at the controller layer before triggering database queries or business operations.
Project Lombok	Used @Getter, @Setter, @Builder, @NoArgsConstructor, and @RequiredArgsConstructor.	Significantly reduces boilerplate code while preserving readable, clean domain models.

2. Angular Frontend Engineering Topics

Angular Topic	Implementation in E-Voting System	Developer Benefit
Standalone Components (Angular 22)	All components use standalone: true with direct imports: [CommonModule, FormsModule, ReactiveFormsModule].	Eliminates legacy NgModule boilerplate. Enables superior tree-shaking, faster esbuild packaging, and self-contained components.
Functional Route Guards (CanActivateFn)	Implemented auth.guard.ts and role.guard.ts checking token validity and role claims (ROLE_ADMIN, ROLE_VOTER).	Prevents unauthorized URL tampering; restricts admin portals and voting machines to authorized credentials.
HTTP Interceptors (HttpInterceptorFn)	Implemented auth.interceptor.ts and error.interceptor.ts via provideHttpClient(withInterceptors([...])).	Automatically injects Authorization: Bearer <token> into outgoing requests and provides global 401/403/500 error interception.

Reactive Forms & Validation	Implemented in voter registration, login, and candidate nomination components with FormBuilder, FormGroup, and custom regex validators.	Provides type-safe form management, reactive input status tracking, and instant UI feedback for invalid voter EPIC numbers.
EVM-Inspired Simulation UI	Dedicated evm-voting component simulating physical voting machine (Candidate rows, symbol, Blue Button, Red LED activation, confirmation modal, and audio beep tone).	Engaging civic-tech user experience replicating real balloting hardware while preventing duplicate button clicks.
RxJS & Observables	Used BehaviorSubject for user session state, pipe(), map(), and catchError() in Angular API services.	Handles asynchronous HTTP communication reactively and propagates authentication state across UI components.
Cascading Master Data Hierarchy	Consumes bihar_master_hierarchy.json to populate multi-level dropdowns (Division -> District -> Block -> Ward).	Eliminates human error by enforcing authentic Bihar and Sitamarhi electoral boundaries.

3. MySQL Database Engineering Topics

MySQL Concept	Implementation in E-Voting System	Data Integrity Guarantee
Database-per-Service Pattern	7 isolated schemas: evoting_auth_db, evoting_voter_db, evoting_election_db, evoting_candidate_db, evoting_voting_db, evoting_result_db, evoting_audit_db.	Absolute domain autonomy. Zero cross-database foreign keys ensure microservice changes never cause database cascade locks.
Composite Unique Constraint	CONSTRAINT uk_election_voter UNIQUE (election_id, voter_id) in voter_ballot_receipts.	Enforces at the InnoDB B-Tree index level that a voter cannot cast two votes in the same election, even under concurrent race conditions.
Secret Ballot Physical Decoupling	Split across voter_ballot_receipts (voter metadata, NO candidate choice) and ballot_votes (candidate choice, NO voter identity).	True cryptographic anonymity: even a Database Administrator with root privileges cannot deduce which candidate a citizen voted for.
Cryptographic Audit Seal	Each vote produces a 64-character SHA-256 hash: SHA256(voterId:electionId:candidateId:timestamp:UUID)	Provides mathematical proof of vote immutability and tamper-

	stored with the ballot.	detection.
InnoDB Engine ACID Guarantees	All tables use ENGINE=InnoDB with row-level locking and write-ahead redo logging.	Guarantees transaction atomicity and prevents dirty reads during high-concurrency polling hours.
2026 Delimitation Master Data	Tables for 9 Divisions, 38 Districts, 17 Blocks of Sitamarhi, 40 Lok Sabha PCs, 243 Assembly ACs, and dynamic ~500 population wards.	Accurately represents official ECI and State Election Commission Bihar electoral delimitations.

4. System Architecture & Interaction Diagrams

Diagram 1: Distributed Microservices Topology

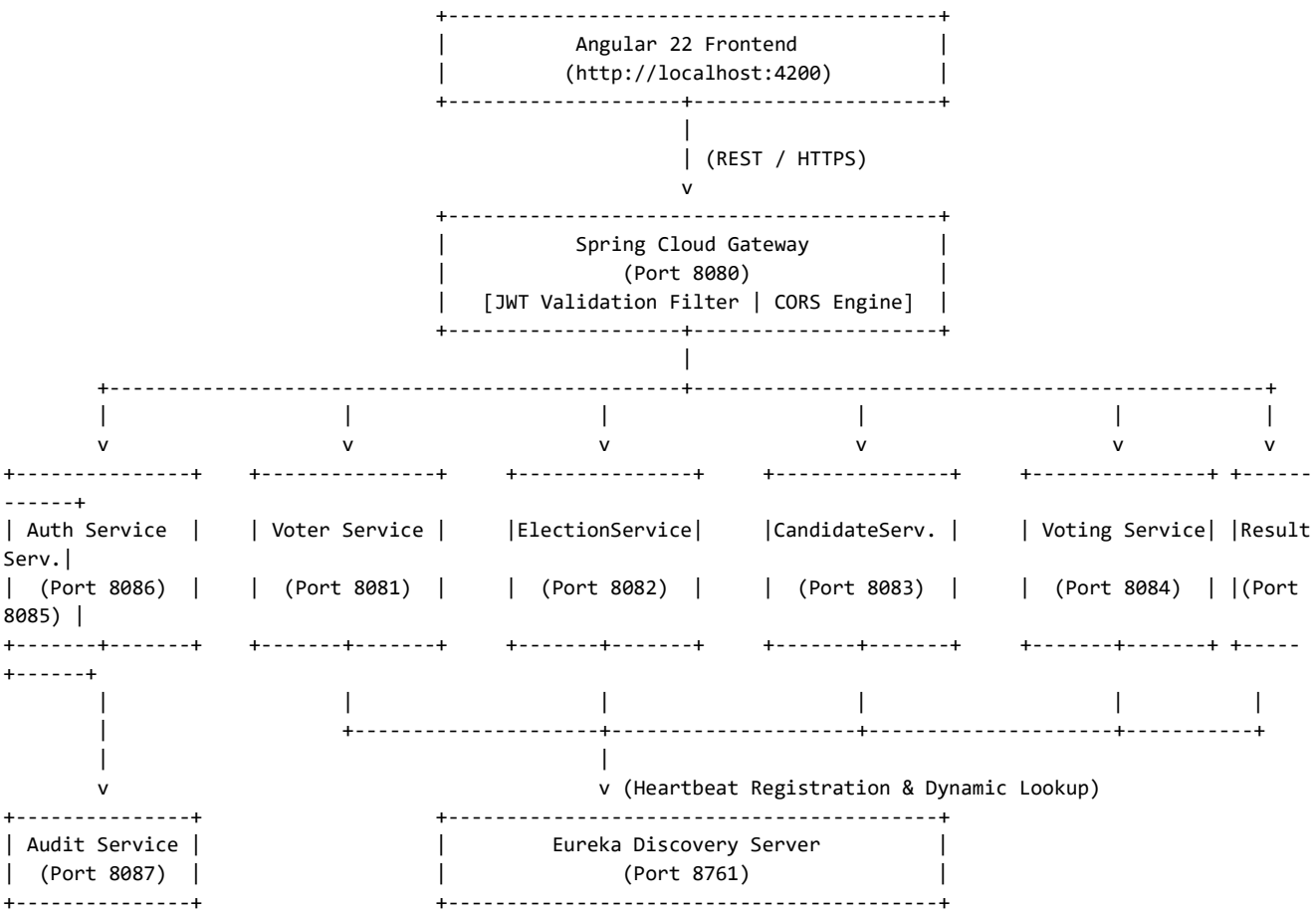
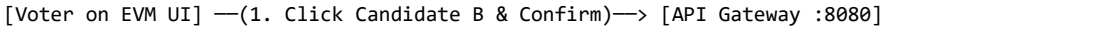
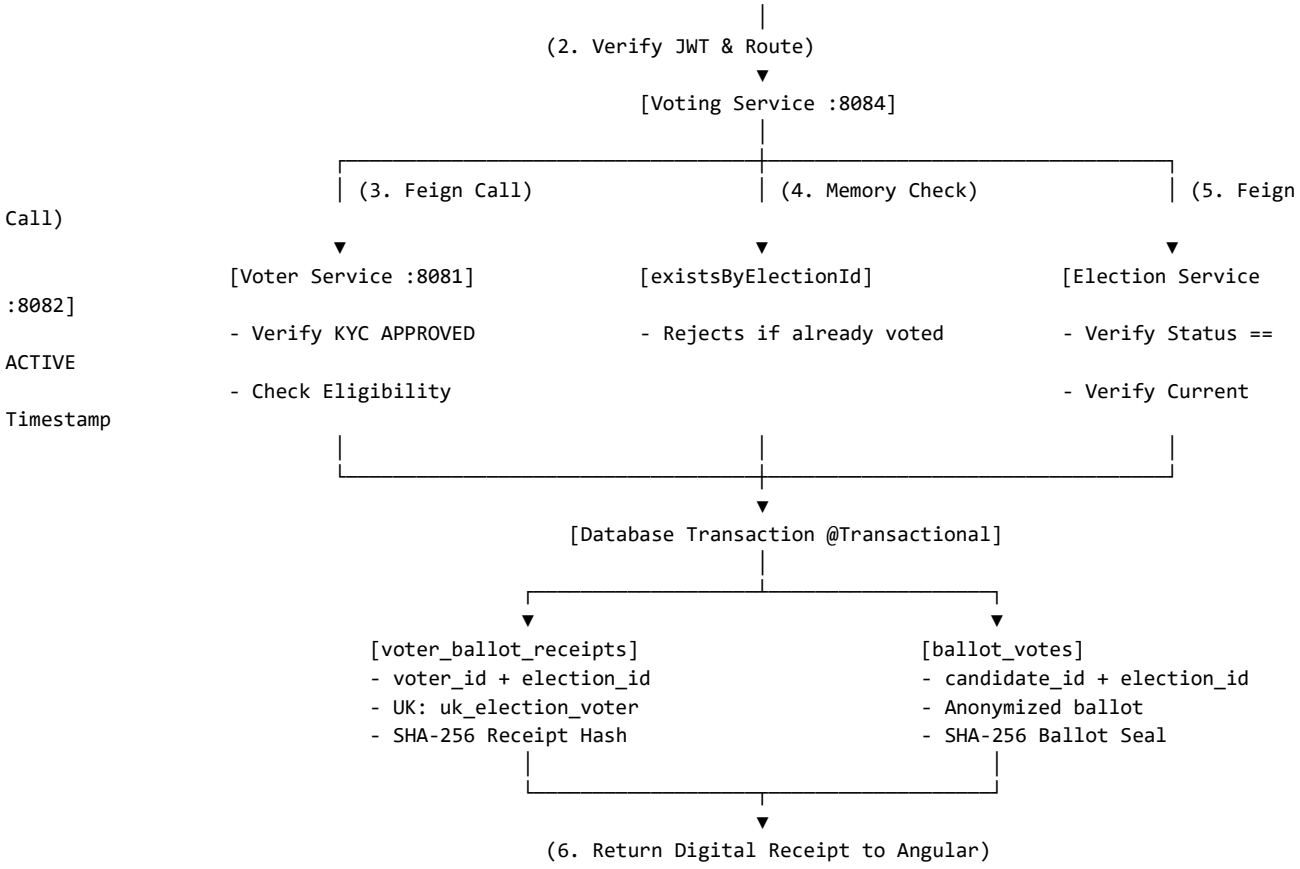


Diagram 2: High-Integrity EVM Vote Transaction Flow





5. Standard Viva Questions & Detailed Answers

Q1: What is the Database-per-Service pattern and why didn't you use a single MySQL database for all microservices?

Short Answer: To ensure domain autonomy, independent scalability, and eliminate tight coupling.

Deep Explanation: In a monolithic application, all features share tables, leading to tight coupling where a schema change in candidate management can break voter authentication. Under the Database-per-Service pattern, each microservice owns its private schema (e.g., evoting_voting_db, evoting_auth_db). No service can execute cross-schema joins. Communication occurs exclusively through defined REST contracts. This also allows individual databases to be scaled, backed up, or migrated independently.

Q2: How does Spring Cloud Netflix Eureka work, and how does it prevent hardcoded IP addresses?

Short Answer: Through dynamic service registration and client-side heartbeat discovery.

Deep Explanation: When a microservice (e.g., voter-service on port 8081) boots up, it contacts the Eureka Server (port 8761) and sends a registration payload containing its service name (VOTER-SERVICE), IP, and port. Every 30 seconds, it sends a heartbeat renewal. When voting-service needs to call voter-service, OpenFeign queries Eureka for live instances of VOTER-SERVICE and resolves the physical URL dynamically using client-side load balancing.

Q3: How does the system technically guarantee 'One-Vote-Per-Voter'?
Short Answer: Dual-defense: application-level query check followed by an authoritative database compound unique constraint.

Deep Explanation: 1. Layer 1 (Application): VotingService calls voteRepository.existsByElectionIdAndVoterId(electionId, voterId). If true, it immediately throws IllegalStateException.
2. Layer 2 (Database Defense): voter_ballot_receipts has a unique constraint: CONSTRAINT uk_election_voter UNIQUE (election_id, voter_id). If two identical requests pass the application check simultaneously (race condition), MySQL's InnoDB engine enforces row uniqueness and aborts the second transaction with a DuplicateKeyException.

Q4: How does Spring Cloud Gateway validate JWT tokens before requests reach downstream services?
Short Answer: Via a reactive Global Gateway Filter that intercepts incoming HTTP headers.

Deep Explanation: The JwtAuthenticationFilter extends Gateway's AbstractGatewayFilterFactory. When a request hits http://localhost:8080/api/votes/cast, the filter extracts the Authorization: Bearer <token> header, uses HMAC-SHA256 secret key to verify cryptographic signature and expiration, checks user roles, and forwards the validated claims down to the microservice. If invalid, it returns HTTP 401 Unauthorized directly at the gateway boundary.

Q5: Why did you choose Angular Standalone Components over NgModules?
Short Answer: Standalone components simplify the mental model, enable better tree-shaking, and remove NgModule configuration overhead.

Deep Explanation: Since Angular 15+, Standalone Components have become the standard. Instead of declaring components inside a centralized NgModule, each component defines its dependencies directly in its @Component({ standalone: true, imports: [...] }) metadata. This makes components truly modular, reusable, easier to unit-test, and allows bundlers like Vite/esbuild to eliminate unused code much more effectively.

6. Tricky & Trap Questions for Senior Full Stack Viva

TRICKY Q1: 'If Voting Service successfully saves the vote in MySQL, but the subsequent Feign call to update Voter Service's hasVoted status fails due to a network glitch, how does the system avoid an inconsistent state?'
Answer: The vote remains valid because Voting Service's database is the authoritative source of truth.

In-Depth Architectural Explanation: In a distributed system, you cannot run a two-phase commit (2PC) easily across microservices without severe performance loss. In our architecture, the voter_ballot_receipts table in voting-service is the Authoritative Source of Truth for who has voted. The hasVoted flag in voter-service is merely a convenience

cache. If a voter attempts to vote again, voting-service checks its own database first and rejects the duplicate immediately, ensuring 100% integrity even if the sync call failed.

TRICKY Q2: 'Can the Database Administrator (DBA) who has full root access to the MySQL server find out who I voted for?'
Answer: No, because voter identity and candidate choices are stored in completely decoupled tables with no foreign key relation.

In-Depth Architectural Explanation: This is the essence of our Secret Ballot Model:
• Table 1: voter_ballot_receipts records (voter_id = 101, election_id = 5, receipt_hash = a8f2...). It proves user 101 voted, but records NO candidate ID.
• Table 2: ballot_votes records (candidate_id = 2, election_id = 5, ballot_hash = 9c4d...). It records the vote, but contains NO voter ID.
• Both records are generated using separate randomized hashes under the same transaction. Therefore, even with complete database root privileges, there is no foreign key or pointer linking Table 1 to Table 2.

TRICKY Q3: 'Why can't we use a JPA @ManyToOne annotation between Vote entity and Candidate entity?'
Answer: Because JPA relationships cannot cross physical microservice and database boundaries.

In-Depth Architectural Explanation: In JPA/Hibernate, @ManyToOne or @JoinColumn assumes that both tables reside in the same relational database schema managed by the same EntityManager. In our microservices architecture, Vote lives in evoting_voting_db (owned by voting-service) while Candidate lives in evoting_candidate_db (owned by candidate-service). Attempting a JPA relationship creates tight database coupling and violates microservice boundary isolation. Instead, we store a scalar foreign identifier (candidateId as a Long) and query details via OpenFeign REST API.

TRICKY Q4: 'What happens if Eureka Server goes down while the voting election is actively running?'
Answer: The system continues running smoothly due to Eureka client-side local registry caching.

In-Depth Architectural Explanation: Eureka clients (Spring Cloud microservices) do not query Eureka for every individual HTTP request. Instead, they maintain a local in-memory cache of the service registry, which is refreshed periodically (default every 30 seconds). If Eureka Server crashes, all microservices continue using their local cached endpoint table to communicate via OpenFeign. As long as existing service instances don't change IP/port, zero downtime occurs.

TRICKY Q5: 'Why should Rajya Sabha election NOT be displayed on ordinary citizens' voter dashboard?'
Answer: Because according to the Constitution of India, Rajya Sabha is an indirect election where only elected MLAs vote.

In-Depth Architectural Explanation: Unlike Lok Sabha (citizens vote for MPs) or Vidhan Sabha (citizens vote for MLAs), Rajya Sabha (Council of States - 16 seats in Bihar) members are elected by an Electoral College of Elected Members of the State Legislative Assembly (MLAs) using Proportional Representation by Single Transferable Vote (STV). Displaying Rajya Sabha balloting to general public voters would be legally and architecturally incorrect. Our system models this in rajya_sabha_representation where public_ballot_allowed = FALSE.

TRICKY Q6: 'Why is Resilience4j configured on Feign clients instead of just letting requests fail?'
Answer: To prevent thread pool exhaustion and cascading system outages.

In-Depth Architectural Explanation: In a microservices architecture, if voter-service slows down (e.g., high database load), every incoming call from voting-service will hold a Tomcat worker thread while waiting for a response timeout. Under high voter concurrency, all worker threads in voting-service become exhausted within seconds, bringing down the entire voting platform. Resilience4j monitors failure rates; when failures exceed 50%, the Circuit Breaker trips to OPEN state and immediately executes a fast fallback without waiting, preserving system stability.